EHB415 ISAC projesi: görev kartları

Bu belge 4 kişilik ekibin bu haftaki işlerini anlatır. Önce **Ortak bilgi**'yi, sonra kendi kartını oku.

Projenin sorusu (herkes bilmeli)

5G baz istasyonu hem kullanıcıya veri taşıyıp hem çevreyi *algılayabilir* (ISAC). Ama kaynaklar sonlu: sensing'e ayrılan kaynak, haberleşmeden gider. **Soru: sensing'e ne kadar kaynak ayırırsak sensing ne kazanır, haberleşme ne kaybeder?** Üç araç bu sorunun üç yüzünü ölçer: **Sionna** kazancı (sensing ne kadar iyi), **ns-3** ve **OCUDU** bedeli (haberleşme ne kadar düşüyor). Hepsi aynı eksende buluşacak: α = **sensing'e giden kaynak payı**.

Herkes için ortak adımlar

1. **Repoyu al:** `git clone https://github.com/tahaberkayalma26-creator/EHB415.git`
2. **Öğreticiyi oku:** repodaki `EHB415_ISAC_Ogretici.pdf`: **Bölüm 1** (sorun), **Bölüm 2.1** (kaynak ızgarası), **Bölüm 6.2** (roller). Yaklaşık 40 dakika.
3. **Kurulumunu yap:** `kurulum/README.md`'deki sıra. Senin kartında hangi kurulum yazıyorsa onu yap. Takılırsan `kurulum/07-sorun-giderme.md`.
4. **Teslim: Perşembe akşamı.** Ayrıntısı kartında.

Üç kural. (1) Sonucu yazarken **hangi komutu çalıştırdığını** yaz; başkası aynısını yapıp aynı sonucu alabilmeli. (2) Sonucun bizimkinden **farklı çıkması sorun değil**: farkı ve nedenini düşündüğünü yaz. (3) Anlamadığın bir şey olursa **teslime "anlamadığım yer" diye yaz**: bu eksik değil, bilgi.

Özet tablo

Kim	Alan	Bu hafta	Çıktı
Taha	3GPP, metrik, rapor	Metrik sözlüğü, α tanımı, rapor	`docs/metrics.md` + rapor taslağı
Nihat	Sionna	Şehir projesini çalıştır, kazanç eğrisini üret	Grafik + 1 paragraf + 3 cevap
Aziz	ns-3 / 5G-LENA	UE taraması, scheduler tablosu, bant deneyi	2 tablo + 1 sayfa not
Fatih	OCUDU	PRS taraması, çok kullanıcılı deney, açık soru	Tablo + sorunun cevabı

Takvim

Bugün: kurulumlar başlar. **Perşembe akşamı:** teslimler; kısa toplantı (ns-3'te sensing yükünü nasıl temsil edeceğimize karar). **Cuma:** rapor yazımı.

Taha: 3GPP standardı, metrik ve rapor

Bu görev neden önemli?

Üç araç farklı şeyler ölçüyor (Sionna: sembol, OCUDU: slot, ns-3: bant). Hepsinin sonucunu **aynı eksene** koymak için **ortak bir "sensing kaynak payı" tanımı** lazım. Bu tanım yoksa üç ayrı, birbiriyle kıyaslanamayan grafik çıkar. Raporun Yöntem bölümü de buna dayanır. Bu kart projenin yapıştırıcısı.

Adımlar

1. **Oku (toplam 2–3 saat).**
   - Lin, *ISAC in 3GPP* (arXiv 2608.11606, 8 sayfa): Tablo 1, bölüm III.A, IV.B.
   - **3GPP TR 38.765 v20.0.0:** §4.1 (metrikler), §4.2 (hedefler), **§6.2 (sensing kaynak oranı)**, §9 (sonuç), Ek A (parametreler). PDF telifli, repoda yok; şuradan indir: `https://whatthespec.net/friendlyspec/pdf/38765/38765-k00.pdf`
   - Dickerson vd. (arXiv 2606.21677): bölüm III.B (denklem 9–12) ve IV.A.
2. **Metrik sözlüğünü yaz:** `docs/metrics.md`. Tablo: *Metrik* | *tanım/formül* | *birim* | *hangi araçta ölçülür*. En az şu satırlar: kaçırma olasılığı P_{md}, yanlış alarm Tip 1 ve Tip 2 (P_{f1}, P_{f2}), konum hatası, hız hatası, throughput, gecikme, paket kaybı, sensing kaynak oranı (Seçenek 1 ve 2).
3. α **tanımını netleştir.** Her araç için bir satır: α *bu araçta nasıl hesaplanıyor?* Şu an: Sionna'da $\alpha = M/140$ (M: 10 ms'lik çerçevede sensing sembolü); OCUDU'da PRS'li slot oranı (ölçülen kayıp $\approx 1/T_{PRS}$); ns-3'te henüz belirsiz (Aziz'in "hangi yol" notuna bağlı; Aziz'le konuş). Hangileri belirsizse işaretle. TR 38.765 §6.2'deki Tip 1/2/3 kaynakları bu tabloya eşle.
4. **Dickerson modelini bir tabloya dök:** $\alpha_{sense} = \frac{N_b}{1-OH} \cdot \frac{f_{sense}}{N_{sym/sn}} \cdot \frac{N_{PRB,sense}}{N_{PRB,toplam}}$ ve $T(\eta) = \min\{\eta, 1-\alpha_{sense}\}$. Makalenin parametreleriyle ($N_{PRB,toplam} = 270$, $OH = 0{,}14$, 14 000 sembol/s, $N_{ZC} = 1291 \Rightarrow 108$ PRB, $f_{sense} = 500$ Hz) kendin hesapla.
5. **Cuma: raporu yaz.** Iskelet: Giriş (boşluk), Arka plan (ISAC, 3GPP), Yöntem (üç araç ve sensing yükünün temsili), Ön bulgular (Nihat, Aziz ve Fatih'in çıktıları), İş planı, Kaynakça.

Kontrol: bizde çıkan sonuç (kendi sonucunla karşılaştır)

Dickerson formülü: tek ışın için $\alpha = \%1{,}66$; sekiz ışın için $\alpha = \%13{,}29$. Makale de aynı değerleri veriyor; tutmuyorsa formülü yanlış yazmışsın.

Ne teslim edeceksin?

(1) `docs/metrics.md`; (2) α tablosu (araç $\times$ tanım $\times$ belirsizlikler); (3) Dickerson hesap tablosu; (4) Cuma: rapor taslağı. **Perşembe akşamına (1)–(3) hazır.**

Nihat: Sionna (kazanç tarafı)

Bu görev neden önemli?

Sionna **"sensing'e kaynak verirsek ne kazanırız?"** sorusunu cevaplar: bir İHA'yı hangi mesafeden, ne kadar doğrulukla buluruz. Ns-3 ve OCUDU yalnızca bedeli ölçer; kazanç olmadan ödünleşim (trade-off) çıkmaz. Ayrıca bu kartın sonunda, filmde gördüğümüz radarın *nasıl çalıştığını* anlamış olacaksın.

Adımlar

1. **Kurulum:** `kurulum/03-sionna-windows.md`, adım 1–6 (Python 3.11 sanal ortam, `sionna==1.2.2`, LLVM, **LLVM için yönetici gerekir**). Bitti: `kurulum\sionna_dogrulama.py` ve `sionna_rt_dogrulama.py` hatasız çalışıyor.
2. **Ek paket:** `python.exe -m pip install pandas` (venv'in python'uyla).
3. **Şehir projesini çalıştır** (PowerShell, repo klasöründe):

```
$env:DRJIT_LIBLLVM_PATH = [Environment]::GetEnvironmentVariable('DRJIT_LIBLLVM_PATH','User')
cd ogretici\sionna_sehir
C:\Users\<kullanici>\venvs\ehb415\Scripts\python.exe rt_sehir.py
C:\Users\<kullanici>\venvs\ehb415\Scripts\python.exe radar_takip.py
```

   `rt_sehir.py` Sionna RT ile şehri ve görüş hattını hesaplar (ilk çalıştırma yaklaşık 1 dk). `radar_takip.py` bu sonucu okuyup radarı simüle eder (yaklaşık 2 dk). Çıktılar `cikti/` klasöründe.
4. **Anla (en önemli adım):** `radar_takip.py` içindeki `radar()` fonksiyonunu satır satır oku. Üç fikir: (i) gecikme, alt taşıyıcılarda *faz dönmesi* yaratır; (ii) alt taşıyıcılar boyunca ters FFT bunu *menzil tepesine* çevirir; (iii) semboller boyunca FFT aynısını *hıza* çevirir. Öğreticide Bölüm 2.4 ve radar atölyesi bunu anlatıyor.
5. **Deney (önce tahmin et, sonra çalıştır):** `radar_takip.py`'nin başındaki `X_SI_DB = 40.0` (artık özgirişim) değerini 30 yap ve yeniden çalıştır. 300 m'deki tespit olasılığı nasıl değişir? *Çalıştırmadan önce yönünü tahmin et ve nedenini yaz.*
6. **Üç soruyu cevapla (her biri 2–3 cümle):**
   (a) Menzil hatası neden sensing payı α'ya bağlı değil, ama hız hatası bağlı?
   (b) Filmde $\alpha = \%10$ ile $\alpha = \%50$ radar görüntüsü neden farklı görünüyor (çizgi ve nokta)?
   (c) İHA binanın arkasına geçince hem $\alpha = \%10$ hem %50 neden hedefi kaybediyor? α'yı artırmak neden kurtarmıyor?

Kontrol: bizde çıkan sonuç (kendi sonucunla karşılaştır)

`rt_sehir.py` sonunda `LOS olan nokta sayisi: 37/60`. `radar_takip.py`'nin tespit olasılığı tablosunda, 300 m'de yaklaşık: $\alpha = \%10$: 0,01; %20: 0,13; %40: 0,49; %50: 0,71; %100: 0,98. 200 m'de $\alpha = \%20$ için yaklaşık 0,99. (Küçük sapmalar normal; rastgele sayı üretici sabit ama kütüphane sürümü farklıysa değişebilir.) Film dosyası: `cikti/05_takip_filmi.gif`.

Ne teslim edeceksin?

(1) Kendi çalıştırmandan `06_kaynak_basari.png` grafiği; (2) "ne gördüm" 3–5 cümle; (3) $X = 30$ dB deneyinin tahmini ve gerçek sonucu; (4) üç sorunun cevabı. Bizim sonuçlardan farklı çıkan bir şey varsa belirt.

Aziz: ns-3 / 5G-LENA (bedel tarafı, ağ düzeyi)

Bu görev neden önemli?

ns-3 **çok kullanıcılı bir ağda** sensing'in bedelini gösterir. Ayrıca projenin en büyük açık sorusu **senin alanında**: 5G-LENA'da hazır bir sensing modeli yok, sensing yükünü simülasyonda *nasıl temsil edeceğimizi* bulman gerekiyor. Raporun Yöntem bölümü bu karara bağlı.

Adımlar

1. **Kurulum:** `kurulum/02-wsl2-ubuntu.md` (WSL2, yönetici gerekir, gerçek yeniden başlatma) sonra `kurulum/04-ns3-5g-lena.md`. Bitti: Ubuntu'da `./ns3 run cttc-nr-demo` komutunu `~/ehb415/ns-3-dev` klasöründe çalıştır; çıktıda `Mean flow throughput` satırı olmalı.
2. **(Önerilir, 30 dk) ns-3'ü tanı:** "iki bilgisayar, bir kablo" programını kendin yaz (referans: `ogretici/betikler/ilk_uygulama_referans.cc`; önce bakmadan yazmayı dene). Çalıştırınca sunucu 2003,69 ms, istemci 2007,37 ms yazmalı. Önce elle hesapla: (1054 bayt × 8 / 5 Mbps) + 2 ms.
3. **UE taraması** (Ubuntu'da, yaklaşık 2 dk):

```
bash ogretici/betikler/ns3_ue_sweep.sh      # cikti: /tmp/ns3_sweep.csv
```

   UE sayısı 1, 2, 5, 10, 20. **100 UE koşma**, 26 dakika sürüyor.
4. **Scheduler tablosu.** `~/ehb415/ns-3-dev/contrib/nr/model/` içinde `nr-mac-scheduler-*` dosyalarını listele. Tablo: *sınıf | algoritma (RR, PF, MR, QoS, random) | zaman (TDMA) ya da frekans (OFDMA) bölüşümü | cttc-nr-demo'da kullanılıyor mu.* İpucu: varsayılan scheduler `contrib/nr/helper/nr-helper.cc` içinde 81. satırda ayarlı.
5. **Bant daraltma deneyi** (bir yoldan sensing'i taklit etmek):

```
cd ~/ehb415/ns-3-dev
./ns3 run "cttc-nr-demo --ueNumPergNb=5 --bandwidthBand1=5e7 --simTime=1"
./ns3 run "cttc-nr-demo --ueNumPergNb=5 --bandwidthBand1=4e7 --simTime=1"
./ns3 run "cttc-nr-demo --ueNumPergNb=5 --bandwidthBand1=2.5e7 --simTime=1"
```

   Her biri için akışların throughput toplamını ve ortalama gecikmeyi yaz. *Önce tahmin et:* bant %20 daralınca ve %50 daralınca throughput ne olur?
6. **"Hangi yol" notu (1 sayfa).** Öğretici Bölüm 4.2.2'deki (a) bant daraltma ve (b) sensing'i ayrı bir trafik akışı olarak ekleme yollarını karşılaştır. Yukarıdaki deney (a)'nın ne zaman işe yaradığını gösteriyor. Hangisini öneriyorsun, neden? (b) için ns-3 alıştırmasındaki ikinci istemci fikrini kullanabilirsin.

Kontrol: bizde çıkan sonuç (kendi sonucunla karşılaştır)

UE taraması (toplam throughput, ortalama gecikme): 1 UE: 10,2 Mbps, 0,28 ms; 2 UE: 112,5, 0,59; 5 UE: 219,7, 9,97; 10 UE: 238,3, 57,9; 20 UE: 288,2, 123,6. **Bant deneyi** (5 UE): 50 MHz: 219,7 Mbps, 9,97 ms; 40 MHz: **219,7** (aynı!), 9,97 ms; 25 MHz: 203,7 Mbps, 19,4 ms. **Dikkat:** 50 → 40 MHz'de hiçbir şey değişmedi. Bu bir hata değil: yük kapasitenin altında olduğunda bandı daraltmak throughput'u etkilemiyor, bedel yalnızca yük kapasiteye yaklaşınca görünüyor. Bu, Dickerson modeli $T(\eta) = \min\{\eta, 1-\alpha\}$ ile aynı şey. **Varsayılan scheduler:** `NrMacSchedulerTdmaRR` (round-robin).

Ne teslim edeceksin?

(1) UE taraması tablosu (kendi çalıştırmandan); (2) scheduler tablosu; (3) bant deneyi sonuçları ve tahminin; (4) 1 sayfa "hangi yol" notu: hangi yolu öneriyorsun, neden, riski ne? **Perşembe karar toplantısında bu notu sen sunacaksın.**

Fatih: OCUDU (bedel tarafı, gerçek kod)

Bu görev neden önemli?

OCUDU **gerçek bir gNB yazılımı**: ns-3 gibi bir model değil, gerçek kod. PRS sinyalinin kaynak maliyetini burada ölçmek sonuçlarımıza güvenilirlik katar. Ayrıca elimizdeki **açık bir teknik soru** senin kartında: 4 sembollük PRS neden 12 sembollük PRS kadar kayıp veriyor? Cevap, "ne kadar kaynak tükettik" sayımızı (TR 38.765'teki Tip 3 kaynak) doğrudan değiştirir.

Adımlar

1. **Kurulum:** `kurulum/02-wsl2-ubuntu.md` (yönetici gerekir) sonra `kurulum/05-ocudu.md`. Bitti: `~/ehb415/ocudu/build/apps/gnb/gnb` programını `--version` ile çalıştır; çıktı `OCUDU 5G gNB version 26.10.0` olmalı.
2. **İki koşu ve tabloyu oku** (Ubuntu'da, repo klasöründe, her biri yaklaşık 35 sn):

```
bash ogretici/betikler/ocudu_prs_kos.sh base 0 0 0       # PRS yok
bash ogretici/betikler/ocudu_prs_kos.sh p4 4 12 104      # PRS her 4 slotta bir
```

   Tablodaki DL `brate` ve `ok` sütunlarına bak. `ok`: saniyede başarılı blok sayısı. *Önce tahmin et:* `ok` 1000'den kaça düşer, neden?
3. **PRS taraması** (yaklaşık 6 dk):

```
bash ogretici/betikler/ocudu_prs_sweep.sh      # cikti: /tmp/prs_sweep.csv
```

4. **Çok kullanıcılı deney:**

```
bash ogretici/betikler/ocudu_cok_ue.sh u4 4 0 0      # 4 UE, PRS yok
bash ogretici/betikler/ocudu_cok_ue.sh u4p4 4 4 0    # 4 UE, PRS her 4 slotta
```

5. **Açık soru (asıl araştırma işi):** *4 sembollük PRS neden 12 sembollük PRS ile aynı kaybı veriyor?* Bakılacak yerler:
   - `~/ehb415/ocudu/lib/scheduler/ue_scheduling/ue_cell_grid_allocator.cpp` ve `grant_params_selector.cpp`: PDSCH için kaynak ararken ızgaranın hangi sembol aralığının doluluğuna bakılıyor?
   - `lib/scheduler/common_scheduling/prs_scheduler.cpp`: PRS ızgaraya `dl_res_grid.fill(grant_info{scs, symbols, crbs})` ile yazılıyor.
   - Log ipucu: PDSCH her zaman `symb=[1, 14)` (slotun tamamı) olarak tahsis ediliyor (kendin görmek için `ogretici/betikler/ocudu_slot_log.sh` betiğini `bash` ile çalıştır).

   Çıktı: bir paragraf cevap. **Bulamazsan da geçerli:** "şu dosyaya, şu fonksiyona baktım, şunu gördüm, şurada takıldım."
6. **(Bonus)** `ogretici/betikler/ocudu_prs_yerlesim.sh` betiğini `bash` ile çalıştır: PRS'i farklı yerlere koymayı dener; hangisini OCUDU reddediyor, nedenleriyle birlikte not al.

Kontrol: bizde çıkan sonuç (kendi sonucunla karşılaştır)

İki koşu: PRS yok: `brate` ≈ 59 M, `ok` = 1000; PRS her 4 slotta: ≈ 44 M, `ok` ≈ 750. **Tarama** (DL Mbps): PRS yok 59,0; $T = 80$: 58,0; $T = 40$: 57,8; $T = 20$: 56,0; $T = 10$: 53,0; $T = 8$: 50,9; $T = 4$: 43,8; $T = 4$ yarım bant (52 PRB): 51,0; $T = 4$, 4 sembol: 43,8. **Çok kullanıcılı:** 4 UE, PRS yok: toplam ≈ 60 Mbps, UE başına ≈ 15; PRS her 4 slotta: toplam ≈ 44, UE başına ≈ 11. **Not:** Bu ölçümler tek koşu ve WSL2 üzerinde (gerçek zamanlı değil). Farklı çıkarsa farkı yaz.

Ne teslim edeceksin?

(1) Tarama tablosu (kendi çalıştırmandan); (2) 4 UE sonucu; (3) açık sorunun cevabı ya da nerelere baktığın; (4) bonus: PRS yerleşim deneyi notu. **Perşembe akşamına (1)–(3) hazır.**